

Kapitel 10

Objektorientierte Programmierung

Die Modellierung eines Ausschnittes der realen Welt geschieht durch eine Klassenhierarchie, d.h., gleichartige Objekte werden zu Klassen zusammengefasst, von denen durch Vererbung Spezialisierungen abgeleitet werden. Gleichartigkeit bedeutet die Übereinstimmung von objektbezogenen Datenfeldern und objektbezogenen Methoden. Eine abgeleitete Klasse erbt von der Oberklasse die dort definierten Datenfelder und Methoden, fügt ggf. eigene hinzu und kann ihnen ggf. durch Überschreiben eine neue Bedeutung geben.

Objekte werden durch Konstruktoren erzeugt. Jede Klasse besitzt einen oder mehrere Konstruktoren, die für das Instanzieren ihrer Objekte zuständig sind. Wird kein eigener Konstruktor definiert, existiert automatisch der vom System bereitgestellte Default-Konstruktor (ohne Parameter).

10.1 Sichtbarkeit von Datenfeldern

Datenfelder, die mit dem Schlüsselwort `static` deklariert werden, heißen *Klassenvariable*. Sie existieren pro Klasse genau einmal (unabhängig von der Zahl der kreierten Instanzen) und alle Objekte dieser Klasse können auf sie zugreifen.

Ein Datenfeld, welches ohne das Schlüsselwort `static` deklariert wird, ist eine sogenannte Instanzvariable. Eine Instanzvariable existiert je Instanz (also je Objekt) genau einmal und kann entsprechend für jede Instanz einen anderen Wert annehmen. Eine Instanzvariable stellt sozusagen eine Eigenschaft eines Objektes dar.

Hinweis: Variablen, die innerhalb einer Methode deklariert werden, bezeichnet man als *lokale Variablen*.

Die Sichtbarkeit von (Instanz- und Klassen-) Variablen und Methoden wird mit Hilfe von Modifiern geregelt. Ist ein Element einer Klasse mit keinem der Schlüsselworte `public`, `private` oder `protected` deklariert, dann ist es nur innerhalb von Klassen desselben Pakets sichtbar.

Unter einem Paket versteht man alle Klassen, die in einem bestimmten Verzeichnis liegen. Sinnvollerweise werden logisch zueinander gehörige Klassen in ein Verzeichnis gepackt. Diese Verzeichnisse werden wiederum logisch zusammengehörig in weitere Verzeichnisse geschachtelt, so dass eine ganze Verzeichnisstruktur entsteht. Nach Java-Konvention sollten die Paketnamen (und damit die Verzeichnisse) immer klein geschrieben sein.

Das Standardpaket besteht aus allen Klassen im aktuellen Arbeitsverzeichnis. Eigene Pakete können beispielsweise angelegt werden, indem man

```
package a.b.c;
```

am Anfang einer Klasse schreibt und diese Klasse auch in dem angegebenen Pfad `a/b/c` ablegt. Eine so definierte Klasse kann nach dem Übersetzen durch

```
java a.b.c.MeineKlasse
```

aufgerufen werden, wobei der Aufrufer sich in dem Verzeichnis befindet, welches das Verzeichnis `a` enthält.

Die Klasse `MeineKlasse` kann von anderen Klassen durch

```
import a.b.c.MeineKlasse;
```

importiert werden.

Die folgende Tabelle zeigt die Umstände, unter denen Klassenelemente der vier Sichtbarkeitstypen für verschiedene Klassen erreichbar sind.

Erreichbar für:	<code>public</code>	<code>protected</code>	<code>paketsichtbar</code>	<code>private</code>
Dieselbe Klasse	ja	ja	ja	ja
andere Klasse im selben Paket	ja	ja	ja	nein
Subklasse in anderem Paket	ja	ja	nein	nein
Keine Subklasse, anderes Paket	ja	nein	nein	nein

- Elemente des Typs `public` sind in der Klasse selbst, in Methoden abgeleiteter Klassen und für den Aufrufer von Instanzen der Klasse sichtbar.
- Elemente des Typs `protected` sind in der Klasse selbst und in Methoden abgeleiteter Klassen sichtbar. Der Aufrufer einer Instanz der Klasse hat nur Zugriff, wenn er in demselben Paket definiert wurde.
- Elemente vom Typ `private` sind nur in der Klasse selbst sichtbar.
- Elemente ohne Modifier gelten als *friendly* und werden als `protected` eingestuft mit der Einschränkung, dass sie in Unterklassen anderer Pakete unsichtbar sind.

10.2 Erste Beispiele

Im Folgenden werden beispielhaft einige Klassen gezeigt, in denen die obigen Konzepte veranschaulicht werden.

```

/***** Datum.java *****/

/** Klasse Datum
 * bestehend aus drei Integers (Tag, Monat, Jahr)
 * und zwei Konstruktoren zum Anlegen eines Datums
 * und einer Methode zur Umwandlung eines Datums in einen String
 */

public class Datum {

    int tag;           // Datenfeld tag
    int monat;         // Datenfeld monat
    int jahr;          // Datenfeld jahr

    public Datum (int t, int m, int j){           // Konstruktor mit 3 Parametern
        tag = t;                                  // initialisiere Tag
        monat = m;                                // initialisiere Monat
        jahr = j;                                  // initialisiere Jahr
    }

    public Datum (int jahr){                       // Konstruktor mit 1 Parameter
        this(1, 1, jahr);                         // initialisiere 1.1. Jahr
    }

    public String toString(){                      // Methode ohne Parameter
        return tag + "." + monat + "." + jahr;    // liefert Datum als String
    }

}

/***** DatumTest.java *****/

import AlgoTools.IO;

/** Klasse DatumTest, testet die Klasse Datum */

public class DatumTest {

    public static void main(String[] argv) {
        Datum d;                                  // deklariere ein Datum
        d = new Datum (15,8,1972);                // kreiere Datum 15.08.1972
        d = new Datum (1972);                     // kreiere Datum 01.01.1972
        d.jahr++;                                  // erhoehe Datum um ein Jahr
        IO.println(d.toString());                 // drucke Datum
        IO.println(d);                            // hier: implizites toString()
    }
}

```

```

/***** Person.java *****/

/** Klasse Person
 * bestehend aus Vorname, Nachname, Geburtsdatum
 * mit einem Konstruktor zum Anlegen einer Person
 * und zwei Methoden zum Ermitteln des Jahrgangs und des Alters
 */

import java.util.GregorianCalendar;

public class Person {

    String vorname;           // Datenfeld Vorname
    String nachname;          // Datenfeld Nachname
    Datum geb_datum;          // Datenfeld Geburtsdatum

    public Person (String vn, // Konstruktor mit Vorname
                  String nn,  // Nachname
                  int t,       // Geburtstag
                  int m,       // Geburtsmonat
                  int j)       // Geburtsjahr
    {
        vorname = vn;          // initialisiere Vorname
        nachname = nn;         // initialisiere Nachname
        geb_datum = new Datum(t,m,j); // initialisiere Geburtsdatum
    }

    public int jahrgang () {   // Methode
        return geb_datum.jahr; // liefert Geburtsjahrgang
    }

    public int alter(){       // Methode
        int jetzt = new GregorianCalendar().get(GregorianCalendar.YEAR);
        return jetzt - geb_datum.jahr; // liefert das Lebensalter
    }

    public String toString() { // Methode, ueberschreibt toString
        return vorname + " " + nachname; // liefert Vor- und Nachname
    }

}

/***** PersonTest.java *****/
import AlgoTools.IO;

/** Klasse PersonTest, testet Klasse Person */

public class PersonTest {

    public static void main (String [] argv) {
        Person p; // deklariere eine Person
        p = new Person("Willi","Wacker",22,8,1972); // kreiere Person
        p.geb_datum.jahr++; // mache sie 1 Jahr juenger
        IO.print(p); // gib Vor- und Nachname aus
        IO.println(" ist "+p.alter()+" Jahre alt."); // gib Alter aus
    }
}

```

```

/***** Student.java *****/

/** Klasse Student, spezialisiert die Klasse Person
 * durch statische Klassenvariable next_mat_nr;
 * durch weitere Datenfelder mat_nr, fach, jsb
 * durch eigenen Konstruktor und durch eigene Methode jahrgang
 * welche die Methode jahrgang der Klasse Person ueberschreibt
 */

public class Student extends Person {           // Student erbt von Person

    static int next_mat_nr = 100000;           // statische Klassenvariable
    int mat_nr;                                // Matrikel-Nummer
    String fach;                               // Studienfach
    int jsb;                                   // Jahr des Studienbeginns

    public Student                               // Konstruktor mit
        (String vn,                             // Vorname
         String nn,                             // Nachname
         int t,                                 // Geburtstag
         int m,                                 // Geburtsmonat
         int j,                                 // Geburtsjahr
         String f,                             // Studienfach
         int jsb)                               // Studienbeginn
    {
        super(vn, nn, t, m, j);                // Konstruktor des Vorfahren
        fach = f;                              // initialisiere Fach
        this.jsb = jsb;                        // initialisiere Studienbeginn
        mat_nr = next_mat_nr++;                // vergib naechste Mat-Nr.
    }

    public int jahrgang() {                     // Methode liefert als Jahrgang
        return jsb;                            // das Jahr des Studienbeginns
    }
}

/***** StudentTest.java *****/

import AlgoTools.IO;

/** Klasse StudentTest, testet die Klasse Student */

public class StudentTest {

    public static void main (String [] argv) {
        Student s;                             // deklariere Student
        s = new Student("Willi","Wacker",22,8,1972,"BWL",1995); // kreiere Student
        IO.print(s);                           // gib Name aus und
        IO.println(" hat die Matrikelnummer " + s.mat_nr); // Matrikelnummer
    }
}

```

```

/***** PersonStudentTest.java *****/

import AlgoTools.IO;

/** Klasse PersonStudentTest
 *  verwendet Instanzen der Klasse Person und der Klasse Student
 */

public class PersonStudentTest {

    public static void main (String [] argv) {

        Student s;                // Student
        Person p;                 // Person

        p = new Person("Uwe", "Meier", 24, 12, 1971); // kreiere Person
        s = new Student("Eva", "Kahn", 15, 9, 1972, "BWL", 1998); // kreiere Student

        IO.println(p + "'s Jahrgang: " + p.jahrgang()); // gib Jahrgang aus
                                                    // da p eine Person ist
                                                    // wird Geburtsjahrgang
                                                    // ausgegeben

        IO.println(s + "'s Jahrgang: " + s.jahrgang()); // gib Jahrgang aus
                                                    // da s ein Student ist
                                                    // wird
                                                    // Immatrikulationsjahr
                                                    // ausgegeben

        p = s;                    // p zeigt auf s

        IO.println(p + "'s Jahrgang: " + p.jahrgang()); // gib Jahrgang aus
                                                    // da p auf Student
                                                    // zeigt, wird
                                                    // Immatrikulationsjahr
                                                    // ausgegeben

        if (p instanceof Student) // falls p Student ist
            IO.println(((Student)p).fach); // gib p's Fach aus

        s = (Student) p;          // s zeigt auf p
        IO.println(s.fach);        // gib s's Fach aus
    }
}

```

Innerhalb einer Vererbungshierarchie können Instanzmethoden überschrieben werden. So wird in der Klasse `Person` die Methode `jahrgang()` definiert. In der Klasse `Student` soll sich diese Methode aber anders verhalten, daher wird sie in der Klasse `Student` überschrieben.

10.3 Binden

Wenn mehrere Methoden mit gleichem Namen und unterschiedlicher Parameterliste existieren, ist die Methode *überladen*.

Statisches Binden bezeichnet den Vorgang, einem Objekt zur Übersetzungszeit den Programmcode fest zuzuordnen; **dynamisches Binden** bezeichnet den Vorgang, einem Objekt zur Laufzeit den Programmcode zuzuordnen.

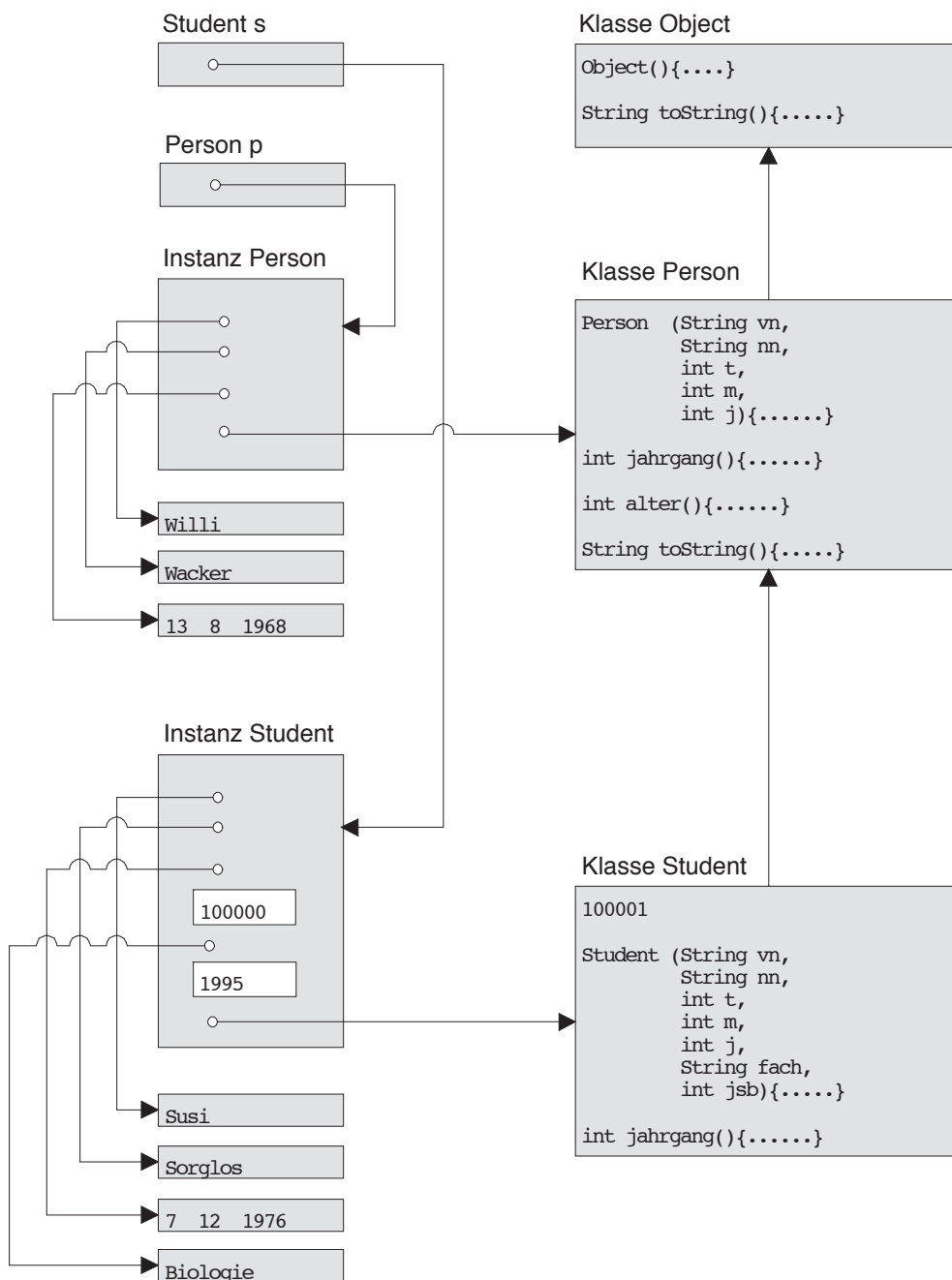
Bei der Programmiersprache Java werden alle Instanzmethoden dynamisch gebunden. Der Compiler prüft lediglich, ob die entsprechende Methode vorhanden ist, aber erst zur Laufzeit wird festgestellt, von welchem Typ das Objekt ist und die entsprechende Methode wird ausgewählt. So kann eine Referenz vom Typ `Person` sowohl auf ein `Person` Objekt verweisen, als auch auf ein `Student` Objekt. Erst zur Laufzeit wird festgestellt, um welches Objekt es sich handelt und die entsprechende Methode wird ausgewählt.

Alles andere (Klassenmethoden, Klassenvariablen, Instanzvariablen) wird statisch gebunden, also bereits zur Compilezeit ausgewählt und *fest verdrahtet*. Der Compiler wählt also die Klassenmethode oder Variable aus, die dem Typ der Referenz entspricht.

Folgende Fälle können z.B. auftreten:

- `Person p = new Person ("Uwe", "Meier", 24, 12, 1971);`
`Student s = new Student ("Eva", "Kahn", 15, 9, 1972, "BWL", 1998);`
Wird übersetzt und zwei Instanzen werden angelegt.
- `IO.println(p.fach);`
Der Compiler versucht das Datenfeld `fach` statisch an das Objekt zu binden. Dies führt zu einem Übersetzungsfehler, da Instanzen vom Typ `Person` nicht über das Datenfeld `fach` verfügen.
- `IO.println(((Student)p).fach);`
Der Compiler versucht das Datenfeld `fach` statisch an das Objekt zu binden. Der Code wird übersetzt, da dem Compiler mit dem Cast vom Programmierer eine Instanz vom Typ `Student` garantiert wird, welche über das Datenfeld `fach` verfügt. Das Beispiel führt aber zu einem Laufzeitfehler, wenn `p` zur Laufzeit nicht auf einen Studenten verweist.
- `p = s;`
Wird übersetzt und ausgeführt. `p` verweist nun auf einen Studenten.
- `IO.println(((Student)p).fach);`
Wird übersetzt und ausgeführt, da `p` nun auf eine Instanz vom Typ `Student` verweist, welche über das Datenfeld `fach` verfügt.
- `IO.println(p.jahrgang());`
Die Methode `jahrgang()` wird dynamisch zur Laufzeit gebunden. Der Quellcode wird übersetzt und ausgeführt. Es kommt die Methode `jahrgang()` aus der Klasse `Student` zum Einsatz. Hinweis: Beim *Statischen Binden* wäre bereits zur Übersetzungszeit dem Objekt `p` die Methode `jahrgang` der Klasse `Person` zugeordnet worden.

Die folgende Grafik verdeutlicht das Zusammenspiel zwischen den Klassen `Person` und `Student` und ihren Instanzen. Auf dem Laufzeitkeller führt von der Variable `s` ein Verweis auf die Speicherfläche, welche eine Instanz vom Typ `Student` aufnimmt. Da einige Bestandteile von `Student` (nämlich `vorname`, `nachname`, `geb_datum` und `fach`) selbst wieder Objekte sind, führen dort hin weitere Verweise; die Instanzvariablen `matr_nr` und `jsb` werden hingegen explizit gespeichert. Ein weiterer Verweis (gezeichnet von links nach rechts) führt zu den Klassenvariablen und Klassenmethoden; von dort sind die Oberklassen erreichbar, von denen geerbt wurde.



10.4 Referenzen

Referenzen auf Objekte modellieren die in konventionellen Programmiersprachen vorhandenen Zeiger. Um dieses Konzept zu demonstrieren, implementieren wir einen Abzählreim in objektorientierter Programmierung. Von n Kindern im Kreis sollen fortwährend jeweils das k -te abgeschlagen werden. Ist Kind Nr. j zur Zeit der Nachfolger von Kind Nr. i , so enthält Kind-Objekt mit Nr. i eine Referenz auf ein Kind-Objekt mit Nr. j .

```

/***** Kind.java *****/
/** Klasse Kind
 * bestehend aus Nummer und Verweis auf Nachbarkind
 * mit Konstruktor zum Anlegen eines Kindes */

public class Kind {
    int nr; // Nummer
    Kind next; // Verweis auf naechstes Kind
    public Kind (int nr) { // Konstruktor fuer Kind
        this.nr = nr; // initialisiere Nummer
    }
}

/***** VerweisAbzaehltreim.java *****/
import AlgoTools.IO;

/** Klasse VerweisAbzaehltreim
 * implementiert einen Abzaehltreim mit k Silben fuer n Kinder mit Verweisen
 * verwendet dabei Objekte vom Typ Kind */

public class VerweisAbzaehltreim {

    public static void main (String [] argv) {

        int i; // Laufvariable
        int n=IO.readInt("Wie viele Kinder ? "); // erfrage Kinderzahl
        int k=IO.readInt("Wie viele Silben ? "); // erfrage Silbenzahl
        Kind erster, letzter, index; // deklarriere drei Kinder
        erster = letzter = new Kind(0); // kreiere erstes Kind

        for (i=1; i < n; i++){ // erzeuge n-1 mal
            index = new Kind(i); // ein Kind mit Nummer i
            letzter.next = index; // erreichbar vom Vorgaenger
            letzter = index;
        }

        letzter.next = erster; // schliesse Kreis
        index = letzter; // beginne bei letztem Kind
        while (index.next != index) { // solange ungleich Nachfolger
            for (i=1; i<k; i++) index=index.next; // gehe k-1 mal weiter
            IO.print("Ausgeschieden: "); // Gib die Nummer des Kindes aus,
            IO.println(index.next.nr, 5); // welches jetzt ausscheidet
            index.next = index.next.next; // bestimme neuen Nachfolger
        }
        IO.println("Es bleibt uebrig: " + index.nr);
    }
}

```

10.5 Wrapperklassen

In der Programmiersprache Java existieren einige *primitive* Datentypen, wie z.B.: `int`, `double`, `boolean`, `char`, usw. Für jeden dieser primitiven Datentypen existiert eine entsprechende Klasse, die einem primitiven Datentyp entspricht. So existiert in Java die Klasse `Integer`, `Double`, `Boolean`, `Character`, usw.

Hinweis: Zeichenketten sind auch Objekte. Diese sind Instanzen der Klasse `String`. Wenn also im Programmcode eine Zeichenkette mit `"Meine Zeichenkette"` definiert wird, erzeugt man indirekt ein neues Objekt, an das auch Methodenaufrufe geschickt werden können.

Zu einem primitiven Datentyp kann man sehr leicht das entsprechende Objekt erhalten. Im folgenden Beispiel werden zwei `Integer` Objekte angelegt, ein Wert inkrementiert und danach beide Werte ausgegeben.

```
int i = 23;
Integer j = new Integer(i);
Integer k = new Integer(42);
j = new Integer(j.intValue()+1);

IO.println(j.intValue());
IO.println(k.intValue());
```

Da hierbei eine Objektstruktur um einen primitiven Datentyp *gewickelt* wird, nennt man diese Klassen auch *Wrapperklassen*.

Will man den Wert eines `Integer` Objektes erhöhen, ist dies, wie im obigen Beispiel deutlich wird, recht umständlich. Dies liegt daran, dass einem Wrapperobjekt kein neuer Wert zugewiesen werden kann. Diese Objekte nennt man auch *immutable*, also unveränderbar.

In Java 5.0 wurde der Umgang mit diesen Wrapperklassen etwas vereinfacht. So ist es nun möglich einer Objektreferenz vom Typ einer Wrapperklasse direkt einen entsprechenden primitiven Datentyp zuzuweisen:

```
int i = 23;
Integer j = i;
Integer k = 42;
j++;

IO.println(j);
IO.println(k);
```

Diese direkte Zuweisung nennt man *Boxing*. Der Wert wird also in die Schachtel des Wrappers gepackt. Umgekehrt kann auch der Wert eines Wrapperobjektes direkt als primitiver Datentyp zurückgeliefert werden. Diesen Vorgang nennt man *Unboxing* und beides zusammen wird als *Autoboxing* bezeichnet.

Diese vereinfachte Schreibweise ist durchaus bequem, man sollte allerdings auch auf die *Nebenwirkungen* achten. Schreibt man in seinem Quellcode z.B.: `Integer j = 23;` wird durch das Compilieren daraus folgender Code: `Integer j = new Integer(23);`.

Implementiert man z.B. eine `for`-Schleife, so könnte man folgendes programmieren:

```
for (Integer i=0;i<1000;i++) {
    IO.println(i.intValue());
}
```

Dieser Quellcode sieht auf den ersten Blick gut aus, allerdings werden hier ständig neue Objekte erzeugt, wodurch das Programm langsamer wird. In Wirklichkeit sieht obige Schleife nämlich wie folgt aus:

```
for(Integer i=new Integer(0);
    i.intValue()<1000;
    i=new Integer(i.intValue()+1)) {
    IO.println(i.intValue());
}
```

Es ist also immer darauf zu achten, wo man Boxing verwendet und ob man sich durch eine vereinfachte Schreibweise ggf. Nachteile einhandelt.

Eine weitere Neuerung in Java 5.0 ist die *for-each*-Schleife. Wird zum Beispiel ein Array von `Integer`-Objekten erzeugt, kann dieses wie gewohnt mit einer `for`-Schleife durchlaufen werden, oder aber mit einer *for-each*-Schleife:

```
Integer [] a = new Integer[10];

for (int i=0; i<10;i++) {           // Array mit Objekten/Werten fuellen
    a[i] = i;                       // Obacht: Boxing
}

for (int i=0; i<10;i++) {
    IO.println(a[i]);
}

for(Integer i: a) {                 // fuer jedes Integer-Objekt aus a
    IO.println(i);                  //
}
```

In der dritten Schleife verweist die Referenz `i` nach und nach einmal auf jedes Objekt im Array.

